

CSE 331: Computer Security

Fundamentals

Spring 2026

Memory Errors

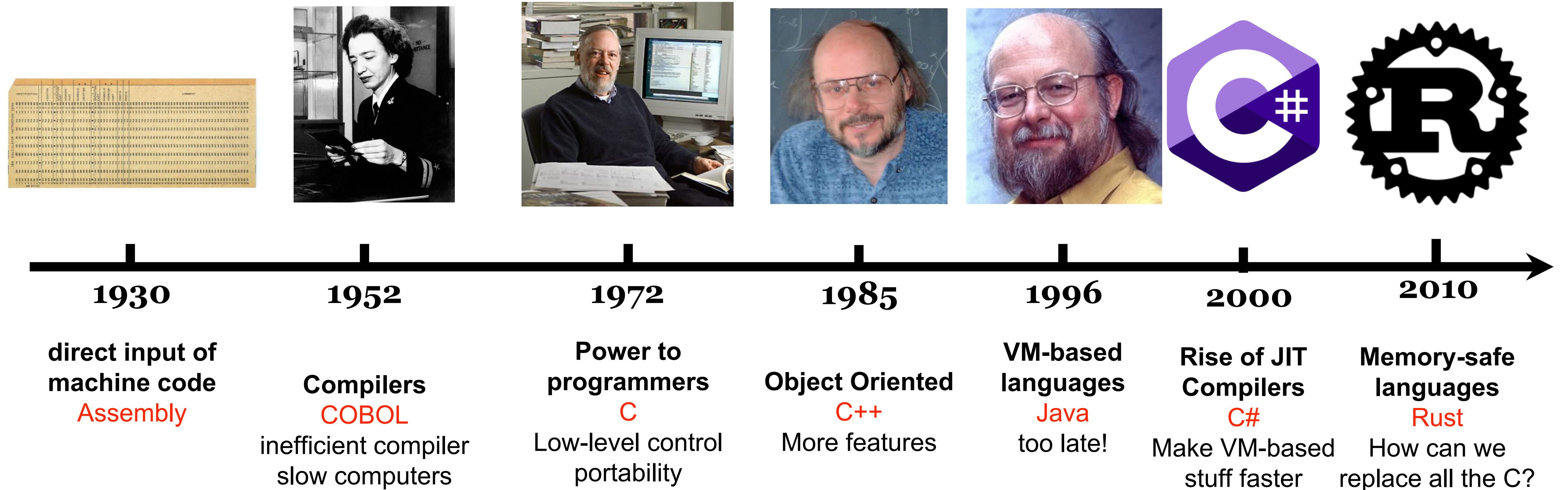
Amir Rahmati
Xinrui Yu



Stony Brook Institute
at Anhui University

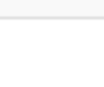
安徽大学纽约石溪学院

Programming Languages



This world of ours

- Everything is written in C.
- Still up there as the most popular PL
- Why can't we just write our OS in Rust?
 - Legacy code
 - Inertia

Aug 2021	Aug 2020	Change	Programming Language		Ratings	Change
1	1			C	12.57%	-4.41%
2	3	▲		Python	11.86%	+2.17%
3	2	▼		Java	10.43%	-4.00%
4	4			C++	7.36%	+0.52%
5	5			C#	5.14%	+0.46%
6	6			Visual Basic	4.67%	+0.01%
7	7			JavaScript	2.95%	+0.07%
8	9	▲		PHP	2.19%	-0.05%
9	14	▲▲		Assembly language	2.03%	+0.99%
10	10			SQL	1.47%	+0.02%

<https://www.tiobe.com/tiobe-index/>

This world of ours

Apr 2026	Apr 2025	Change	Programming Language		Ratings	Change
1	1			Python	20.97%	-2.11%
2	3	^		C	12.34%	+2.39%
3	2	v		C++	8.03%	-2.30%
4	4			Java	7.79%	-1.84%
5	5			C#	5.98%	+1.59%
6	6			JavaScript	3.11%	-0.60%
7	8	^		Visual Basic	3.02%	+0.08%
8	10	^		SQL	1.75%	-0.44%
9	14	^^		R	1.62%	+0.43%
10	9	v		Delphi/Object Pascal	1.52%	-1.01%

What's wrong with C?

Problem 1: Trusting the developers

- In Python:

```
>>> a = [ 1, 2, 3 ]  
>>> print a[10] = 0x41;  
IndexError: list index out of range
```

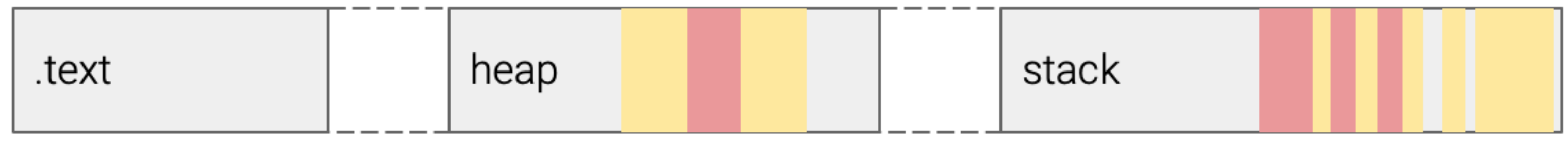
- In C:

```
int a[3] = { 1, 2, 3 };  
a[10] = 0x41;  
// no problem!
```

```
int a[3] = {1, 2, 3};  
if (index >= 0 && index < 3) {  
    a[index] = value;  
}
```

- Direct translation! – C gives you Direct Memory Access (DMA).
- If you want to fix it, write a structure yourself!

What's wrong with C?



- Programs start up with potentially user-influenced data already present:
- C expect to user a universal pointer for everything: code, data, stack, heap...
- During execution, user data spreads through the program:
- User data shouldn't directly control program execution. It is normally "non-control" data. However, it is stored together with "control" data.

Problem 2: Mixing control information and data

Remember the stack?



- Everything is jumbled together...
 - local variables of the active function
 - saved pointers to other places on the stack or to data in memory
 - saved pointers to code (return addresses)
 - local variables of the caller function (and its caller function and so on)
- All of this data is stored together and treated the same...

```
int a[3] = { 1, 2, 3 };  
a[10] = 0x41;
```


Example:

- What happens if you try to write past the end of an array in C?
`int a[3] = { 1, 2, 3 };
a[10] = 0x41;`
- The compiler does not stop you. The program runs without immediate error.
- You overwrite memory that does not belong to the array. This extra memory could contain:
 - Other local variables.
 - Function pointers.
 - Return addresses.
 - Stack canaries (if protections are enabled).
 - Or even unrelated data in memory.

What's wrong with C?

Problem 2: Mixing control information and data



- Memory corruption occurs when user-controlled data manages to spread into data that shouldn't be user controlled (through a memory error).
- If you overwrite control data (i.e., a return address), you can use this to redirect control flow elsewhere.
- Or you can also redirect it to your injected code.

Morris Worm

The worm **overwrote the return address** on the stack with attacker-controlled data, redirecting execution to malicious code

What's wrong with C?

Problem 3: Mixing metadata and data

Metadata is data about data that helps to organize, find and understand data.

- Strings are null terminated in C.

```
char name[10] = "Amir";
```

A	m	i	r	\0	\0	\0	\0	\0	\0
---	---	---	---	----	----	----	----	----	----

- 4 bytes are data ("Amir"), one (the first NULL byte) implicitly encodes the length of the data.
- What if you want to do a read?

```
read(0, name, sizeof(name));
```

Truncating user input

- What if there are NULL bytes in the input?

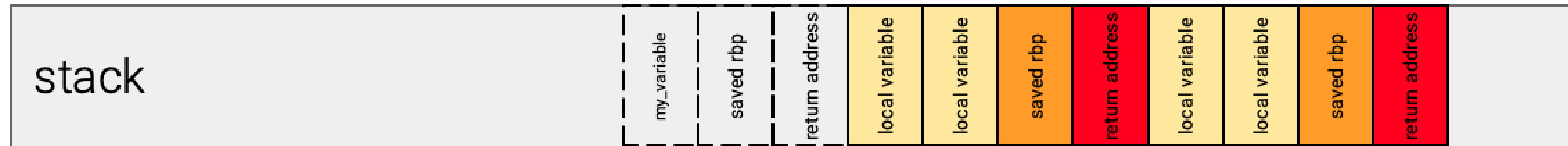
A	m	i	r	R	\0	h	m	a	t
---	---	---	---	---	----	---	---	---	---

- What if there are no NULL bytes in the input?

A	m	i	r	R	a	h	m	a	t
---	---	---	---	---	---	---	---	---	---

Reading beyond buffer limit

What's wrong with C?



- If you don't clean before/up after yourself, C won't do it for you!

```
void my_function()  
{  
    char my_variable[8];  
    // what is the value of my_variable here?  
}
```

- Without cleanup, sensitive information can be leaked.

Problem 4: Initialization and Cleanup

Memory Errors

```
01 int main(int argc, char **argv, char **envp)
02 {
03   print_quoted(argv[1]);
04   return 0;
05 }
06 void print_quoted(char *s)
07 {
08   printf(quote(s));
09   return;
10 }
11 char *quote(char *s) {
12   char output[16];
13   sprintf(output, "\"%s\"", s);
14   return output;
15 }
```

16-byte buffer (unchecked)



Memory Errors

```
01 int main(int argc, char **argv, char **envp)
02 {
03   print_quoted(argv[1]);
04   return 0;
05 }
06 void print_quoted(char *s)
07 {
08   printf(quote(s));
09   return;
10 }
11 char *quote(char *s) {
12   char output[16];
13   sprintf(output, "\"%s\"", s);
14   return output;
15 }
```

sprintf Write formatted data to string without checking the buffer



Memory Errors

```
01 int main(int argc, char **argv, char **envp)
02 {
03   print_quoted(argv[1]);
04   return 0;
05 }
06 void print_quoted(char *s)
07 {
08   printf(quote(s));
09   return;
10 }
11 char * quote(char *s) {
12   char output[16];
13   sprintf(output, "\"%s\"", s);
14   return output;
15 }
```

Buffer Overflow: Overwrite memory at lower addresses



Two things going wrong here:

1. Lazy/insecure programming practices (gets, strcpy, scanf, sprintf, etc).
2. Passing pointers around without their size. Even if we wanted to use a safe function (such as snprintf), there was not enough information!

```
01 int main(int argc, char **argv, char **envp)
02 {
03     print_quoted(argv[1]);
04     return 0;
05 }
06 void print_quoted(char *s)
07 {
08     printf(quote(s));
09     return;
10 }
11 char * quote(char *s) {
12     char output[16];
13     sprintf(output, "\"%s\"", s);
14     return output;
15 }
16 void win() { sendfile(1, open("/flag", O_RDONLY), 0, 128)); }
```

snprintf:

If the resulting string would be longer than $n-1$ characters, the remaining characters are discarded and not stored

Prints the first 128 bytes of the /flag file to the terminal or console. Attacker win condition

What can we corrupt?

1. Memory that doesn't influence anything.
 2. Memory that is used in a **value** to **influence mathematical operations**, conditional jumps, etc (such as the win variable).
 3. Memory that is used as a **read pointer** (or offset), allowing us to **force the program to access arbitrary memory**.
 4. Memory that is used as a **write pointer** (or offset), allowing us to **force the program to overwrite arbitrary memory**.
 5. Memory that is used as a **code pointer** (or offset), allowing us to **redirect program execution**. —> Control flow hijack
- Typically, you use one or more vulnerabilities to achieve multiple of these effects.

What can we corrupt?

6. Memory that is used as the **return address** of a function during program execution to **control what is executed next**.

Lets you jump to arbitrary functions.

Lets you jump to arbitrary instructions.

Lets you chain functionality.

(On x86 and amd64) lets you jump between instructions.

Memory Errors

```
01 int main(int argc, char **argv, char **envp)
02 {
03   print_quoted(argv[1]);
04   return 0;
05 }
06 void print_quoted(char *s)
07 {
08   printf(quote(s));
09   return;
10 }
11 char * quote(char *s) {
12   char output[16];
13   sprintf(output, "\"%s\"", s);
14   return output;
15 }
```



Memory Errors

```
01 int main(int argc, char **argv, char **envp)
02 {
03   print_quoted(argv[1]);
04   return 0;
05 }
06 void print_quoted(char *s)
07 {
08   printf(quote(s));
09   return;
10 }
11 char * quote(char *s) {
12   char output[16];
13   sprintf(output, "\"%s\"", s);
14   return output;
15 }
```



Memory Errors

```
01 int main(int argc, char **argv, char **envp)
02 {
03     print_quoted(argv[1]);
04     return 0;
05 }
06 void print_quoted(char *s)
07 {
08     printf(quote(s));
09     return;
10 }
11 char * quote(char *s) {
12     char output[16];
13     sprintf(output, "\"%s\"", s);
14     return output;
15 }
16 void win() { sendfile(1, open("/flag", O_RDONLY), 0, 128)); }
```

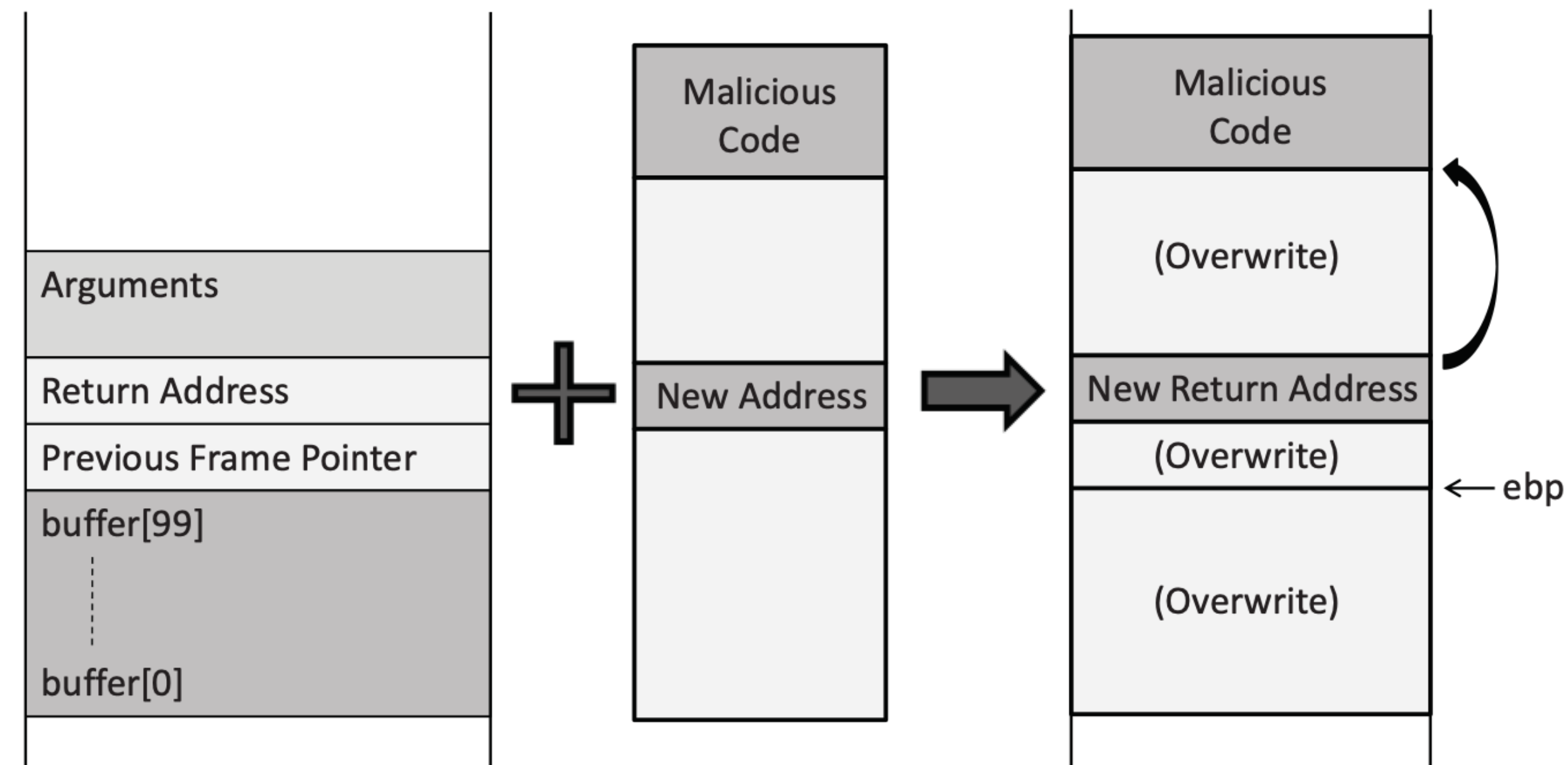


Cause of Corruption

Classic Buffer Overflow Small buffer, large user input

- Because C does not implicitly track buffer sizes, simple overwrites are common.
- Smallest possible example:

```
int main(int argc, char **argv, char **envp)
{
    char buffer[100];
    read(0, buffer, 256);
}
```



Cause of Corruption

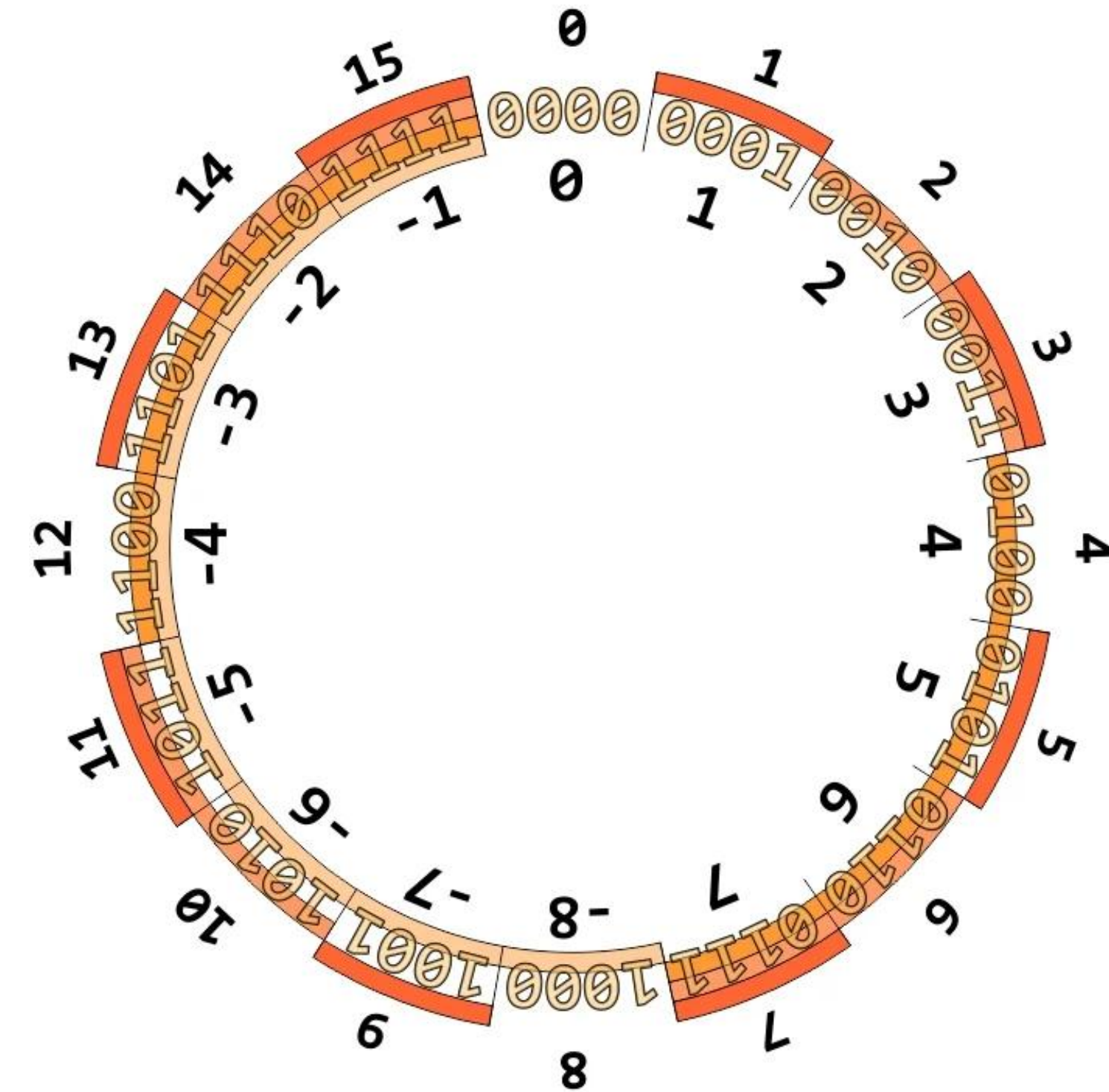
Signedness Mixups

- The standard C library uses unsigned integers for sizes (i.e., the last argument to read, memcmp, strncpy, and others). The default integer types (short, int, long) are signed.

```
int main() {  
    int size;  
    char buf[16];  
    scanf("%i", &size);  
    if (size > 16) exit(1);  
    read(0, buf, size);  
}
```

User inputs -1

- Why is this a problem? Recall twos complement:
 - 0xffffffff == -1, 0xffffffe == -2, etc
 - signedness mostly matters during conditional jumps
 - cmp eax, 16; jae too_big
 - unsigned comparison
 - eax = 0xffffffff will result in checking 0xffffffff > 16 and a jump
 - cmp eax, 16; jge too_big
 - signed comparison
 - eax = 0xffffffff will result in checking -1 > 16, and no jump
- Read uses unsigned



Cause of Corruption

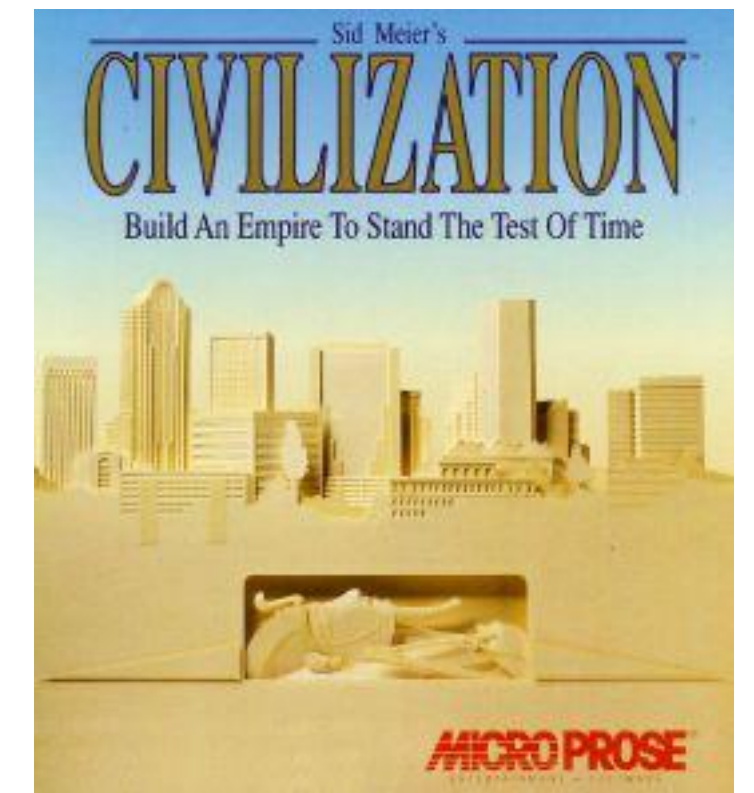
Integer Overflows

- When developers try to calculate sizes, mistakes can occur...
- What's the maximum value that a 32-bit integer can take?
- What happens when you increment that?

```
int main() {  
    unsigned int size;  
    scanf("%i", &size);  
    char *buf = alloca(size+1);  
    int n = read(0, buf, size);  
    buf[n] = '\0';  
}
```


Cause of Corruption

Integer Overflows



1991



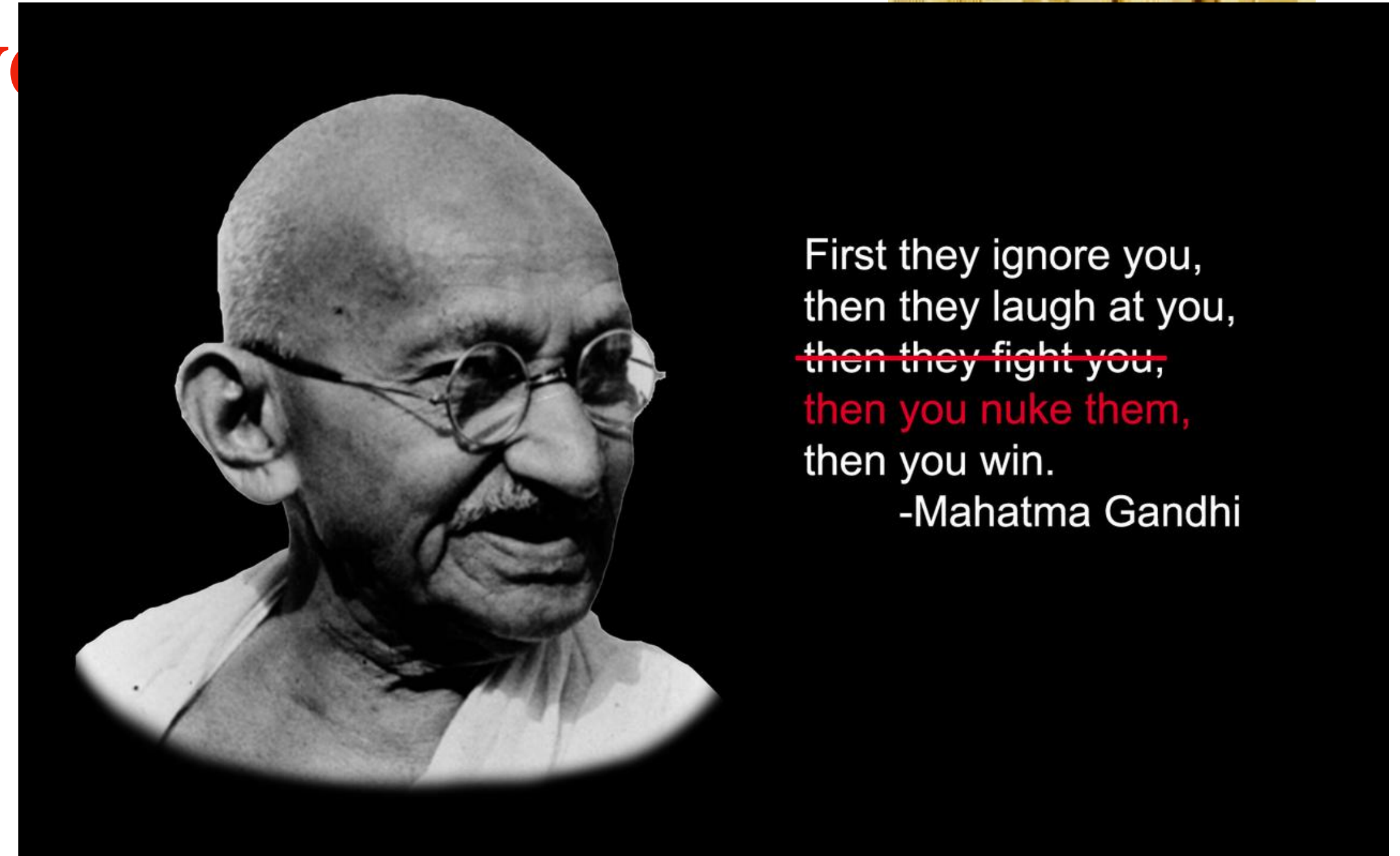
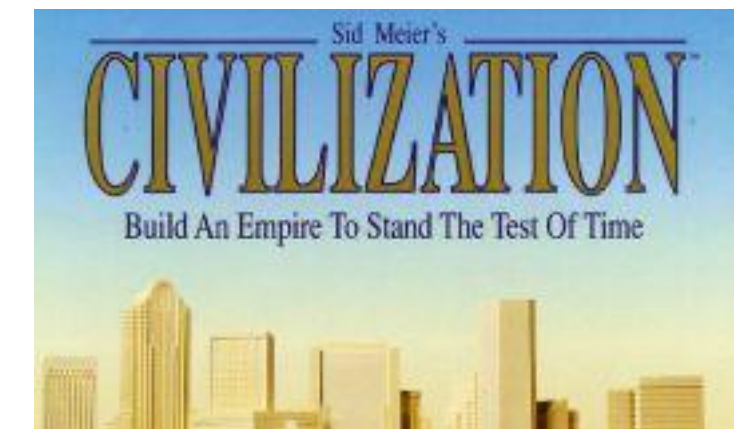
2025

Cause of Corruption

Integer Overflow

- Each leader's AI had an aggression value between 1-10.
- Gandhi was the only one with 1
- Changing government type to democracy reduced aggression by 2
- value was stored in 1 byte unsigned int.
- **255** aggression
— NUCLEAR GANDHI

Well, actually an urban legend.



	Gandhi	Genghis Khan	Gustavus Adolphus	Haile Selassie	Harald Bluetooth	Harun al-Rashid	Hiawatha	Isabella
Trade Origin	5	5	5	5	5	5	5	5
Military Training	3	6	8	4	6	5	4	5
Mobile	7	8	7	5	5	6	4	5
Naval	3	5	5	3	8	4	3	7
Naval Growth	3	5	5	3	8	4	4	7
Naval Recon	3	5	3	3	8	4	3	8
Naval Tile Improvement	3	5	5	3	8	4	4	7
Build Nuke	12	7	5	5	5	5	3	5
Offense	4	8	6	4	7	4	3	6
Production	5	6	6	6	5	5	5	5
Long Range	7	7	4	5	4	7	7	5
Recon	4	6	4	5	5	5	6	6
Religion	8	3	4	7	5	8	7	8
Science	6	4	7	4	5	7	4	3
Spaceship	7	5	5	5	5	8	8	5
Tile Improvement	5	5	5	5	5	6	4	4
Use Nuke	12	7	5	5	5	x	5	x

Facts Regarding Nuclear Gandhi

- In reality, according to the Civilization II lead game designer Brian Reynolds, based on his memories of Civilization's source code, Reynolds stated that there was no unsigned variable in this section of code and that leaders could not act more aggressively than the most aggressive leaders of the game.
- A leader with an aggression level of 255 would act the same way as a leader with an aggression level of 3.
- According to Sid Meier, since all integer variables are signed by default in both C and C++ (the programming languages of Civilization and Civilization II respectively), overflow would not have occurred if Gandhi's aggression were set to -1 .

Integer Overflow Happens a Lot



- In Europa Universalis IV, casualties number can overflow, resulting in negative number of soldiers.

Cause of Corruption

Off-by-one Errors



```
int a[3] = { 1, 2, 3 };  
for (int i = 0; i <= 3; i++) a[i] = 0;
```

- Off-by-one errors can cause small amounts of memory corruption.
- Depending on what you corrupt in memory, this can be disastrous.

Defenses

Stack Canary

StackGuard: Automatic Adaptive Detection and Prevention of Buffer-Overflow Attacks*

Crispin Cowan, Calton Pu, Dave Maier, Heather Hinton[†], Jonathan Walpole,
Peat Bakke, Steve Beattie, Aaron Grier, Perry Wagle and Qian Zhang
Department of Computer Science and Engineering
Oregon Graduate Institute of Science & Technology
immunix-request@cse.ogi.edu, <http://cse.ogi.edu/DISC/projects/immunix>

Abstract

This paper presents a systematic solution to the persistent problem of buffer overflow attacks. Buffer overflow attacks gained notoriety in 1988 as part of the Morris Worm incident on the Internet. While it is fairly simple to fix individual buffer overflow vulnerabilities, buffer overflow attacks continue to this day. Hundreds of attacks have been discovered, and while most of the obvious vulnerabilities have now been patched, more sophisticated buffer overflow attacks continue to emerge.

We describe StackGuard: a simple compiler technique that virtually eliminates buffer overflow vulnerabilities with only modest performance penalties. Privileged programs that are recompiled with the StackGuard compiler extension no longer yield control to the attacker, but rather enter a fail-safe state. These programs require no source code changes at all, and are binary-compatible with existing operating systems and libraries. We describe the compiler technique (a simple patch to `gcc`), as well as a set of variations on the technique that trade off between penetration resistance and performance. We present experimental results of both the penetration resistance and the performance impact of this technique.

*This research is partially supported by DARPA contracts F30602-96-1-0351 and F30602-96-1-0302.
[†]Ryerson Polytechnic University

1 Introduction

This paper presents a systematic solution to the persistent problem of buffer overflow attacks. Buffer overflow attacks gained notoriety in 1988 as part of the Morris Worm incident on the Internet [23]. Despite the fact that fixing individual buffer overflow vulnerabilities is fairly simple, buffer overflow attacks continue to this day, as reported in the SANS Network Security Digest:

Buffer overflows appear to be the most common problems reported in May, with degradation-of-service problems a distant second. Many of the buffer overflow problems are probably the result of careless programming, and could have been found and corrected by the vendors, before releasing the software, if the vendors had performed elementary testing or code reviews along the way.[4]

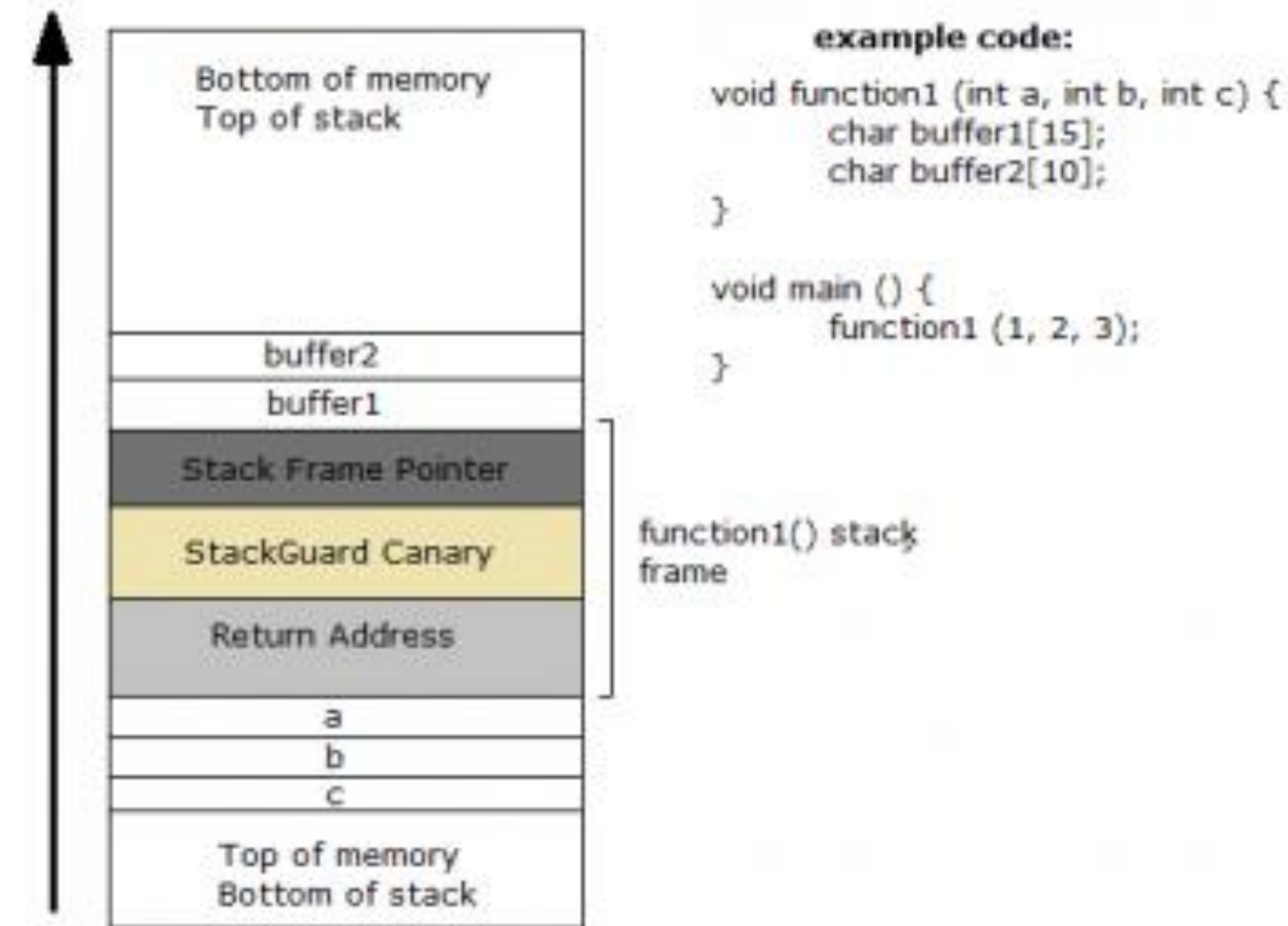
The base problem is that, while individual buffer overflow vulnerabilities are simple to patch, the vulnerabilities are profligate. Thousands of lines of legacy code are still running as privileged daemons (SUID `root`) that contain numerous software errors. New programs are being developed with more care, but are often still developed using unsafe languages such as C, where simple errors can leave serious vulnerabilities.

The continued success of these attacks is also due to the "patchy" nature by which we protect against such attacks. The life cycle of a buffer overflow attack is simple: A (malicious) user finds the vulnerability in a highly priv-



Stack Canary

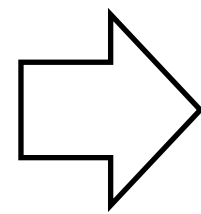
- In function prologue, write random value at the end of the stack frame.
- In function epilogue, make sure this value is still intact.
- Canary value can be:
 - Random value (choose once for whole process)
 - NULL bytes / EOF / etc.
(string functions won't copy past canary)



How does it work?

```
char *func(char *msg) {
    int var1;
    char buf[80];
    int var2;
    strcpy(buf,msg);
    return msg;
}

int main(int argv, char **argc)
{
    char *p;
    p = func(argc[1]);
    exit(0);
}
```



function_prologue:

```
    pushl $0x000aff0d // push canary into the
stack
    pushl %ebp // save frame pointer
    mov %esp,%ebp // saves a copy of current %esp
    subl $108, %esp // space for local variables

    (function body)
```

function_epilogue:

```
    leave // standard epilogue
    cmpl $0x000aff0d, (%esp) // check canary
    jne canary_changed
    addl $4,%esp // remove canary from stack
    ret

canary_changed: ... // abort the program with
error
    call __canary_death_handler
    jmp . // just in case I guess
```

Stack Canary

- Stack canaries are VERY effective in general.
- Situational bypass methods:
 - Leak the canary (using another vulnerability).
 - Brute-force the canary (for forking processes, like lockpicking).

```
int main() {  
    char buf[16];  
    while (1) {  
        if (fork()) { wait(0); }  
        else { read(0, buf, 128); return; }  
    }  
}
```

- Jumping the canary (if the situation allows).

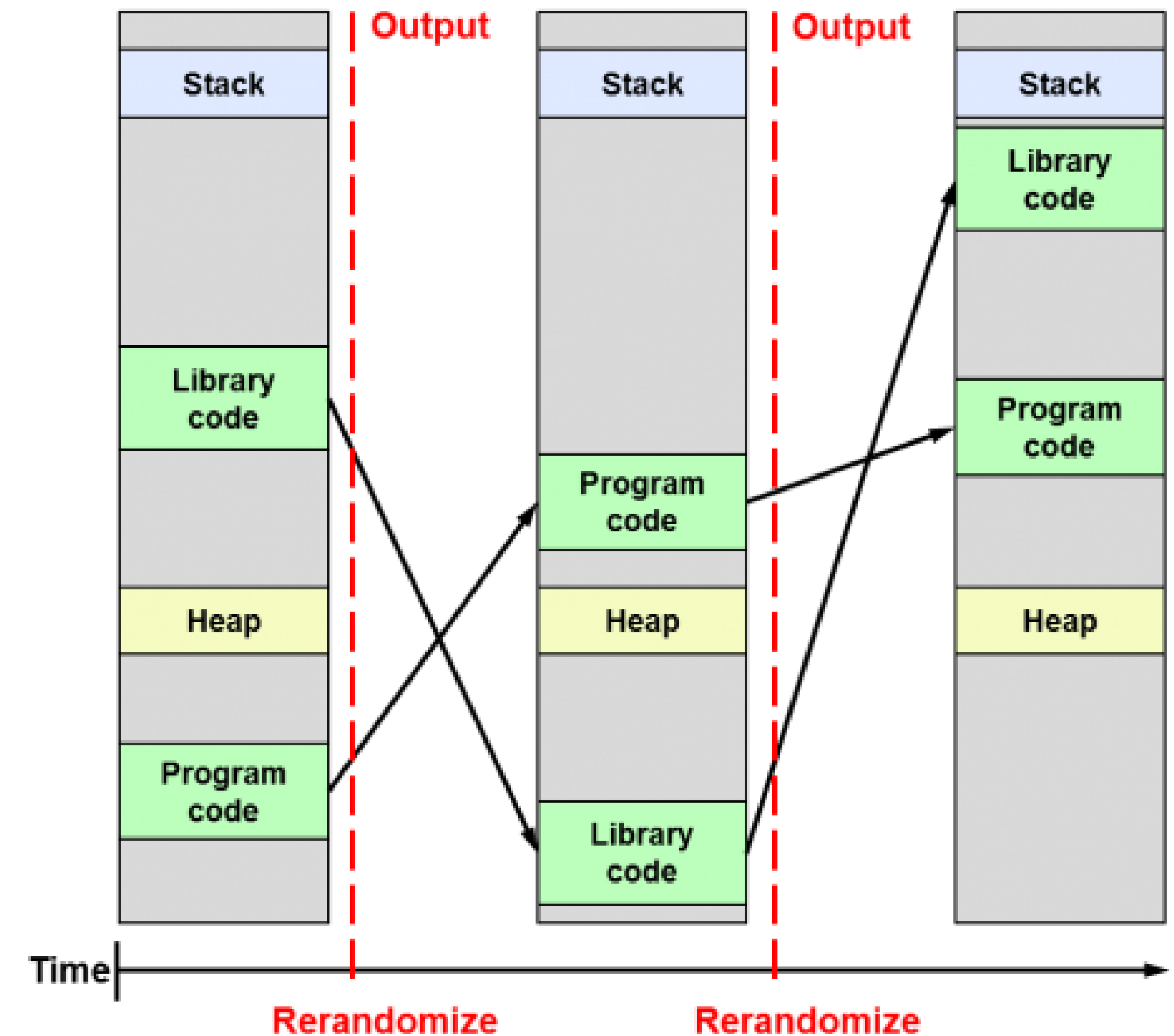
```
int main() {  
    char buf[16];  
    int i;  
    for (i = 0; i < 128; i++) read(0, buf+i, 1);  
}
```

Depending on the stack layout, you can overwrite i and redirect the read to point to after the canary!

Defenses

Address Space Layout Randomization

- Memory corruption often focuses on corrupting pointers to point somewhere else.
- What if we randomize the location of code and data in memory? What can the attacker point to?



ASLR

Workarounds

- How do you redirect execution if you don't know where any code is?
- **Method 1: Leak**
 - The addresses still (mostly) have to be in memory so that the program can find its own assets.
 - Leak them out!
- **Method 2: YOLO**
 - Program assets are page-aligned.
 - Overwrite just the page offset!
 - Requires some brute-forcing.
- **Method 3 (situational): brute-force**

```
int main() {  
char buf[16];  
while (1) {  
    if (fork()) { wait(0); }  
    else { read(0, buf, 128); return; }  
}  
}
```

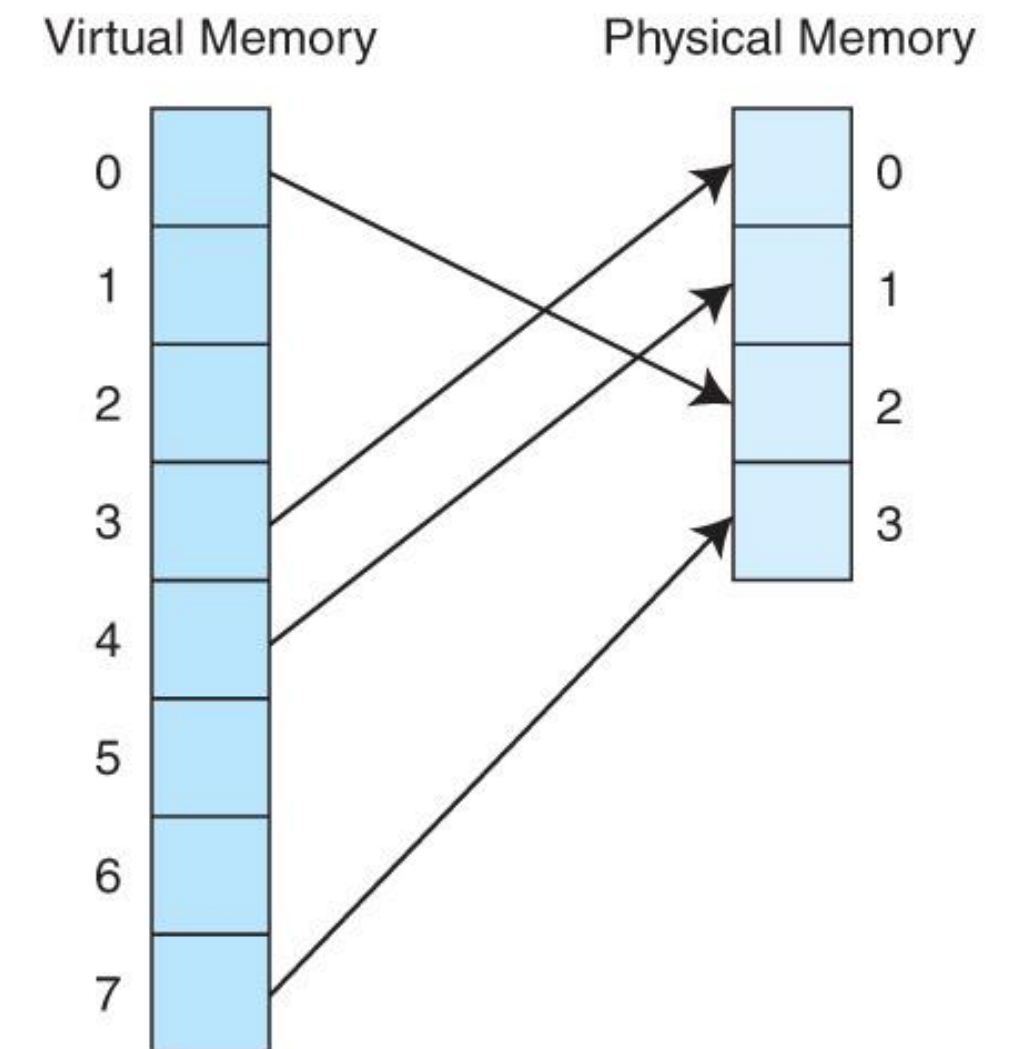
ASLR

Overwriting Page Offsets

- Pages are always aligned to a 0x1000 alignment.
- Possible page addresses:

0x00007f8dce27f000 0x56531c9c5000 0xffffffff600000 0x400000

- The last three nibbles of an address are never changed.
- If we overwrite the two least significant bytes of a pointer, we only have to brute-force one nibble (16 possible values) to successfully redirect the pointer to another location on the same page.
- With little endian, these are the first two bytes that we will overwrite!



ASLR

Disabling ASLR for local testing

- In pwntools:

```
pwn.process("./vulnerable_program", aslr=False)
```

- gdb will disable ASLR by default if it has permissions to do so.
 - For SUID binaries, remove the SUID bit before using gdb (chmod or cp).
- You can spin up a shell whose (non-setuid) children will all have ASLR disabled:

```
# setarch x86_64 -R /bin/bash
```


Causes of Disclosure

Buffer Overread

- Analogous to a buffer overflow!

```
int main(int argc, char **argv, char **envp)
{
    char small_buffer[16] = {0};
    write(1, small_buffer, 128);
}
```

- What is written?

00000000	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	
00000010	c0 82 8f e8 fc 7f 00 00	00 d9 21 50 55 03 e8 37	!PU..7
00000020	00 00 00 00 00 00 00 00	b3 a0 57 92 ae 7f 00 00	W....
00000030	20 26 79 92 ae 7f 00 00	c8 82 8f e8 fc 7f 00 00	&y.....
00000040	00 00 00 00 01 00 00 00	69 61 fc 07 3a 56 00 00	ia.:V..
00000050	e0 61 fc 07 3a 56 00 00	8a 16 b7 d7 2a b6 e1 73	.a.:V.....*.s
00000060	80 60 fc 07 3a 56 00 00	c0 82 8f e8 fc 7f 00 00	.`.:V.....
00000070	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	

Causes of Disclosure

Termination Problems

- Like many things in C, strings do not have explicit size metadata in memory.
- To solve this, they are null-terminated.

```
char name[5] = "Amir";  
// stored as hex 41 6d 69 72 00
```

- Problem: people often forget the null byte (note the conceptual relationship to off-by-one)!

```
int main() {  
    char name[10] = {0};  
    char flag[64];  
    read(open("/flag", 0), flag, 64);  
    printf("Name: ");  
    read(0, name, 10);  
    printf("Hello %s!\n", name);  
}
```

Causes of Disclosure

Uninitialized Data

- Recall that C will not clean up for you!

```
int main() { foo(); bar(); }  
void foo() { char foo_buffer[64]; read(open("/flag", 0), foo_buffer, 64); }  
void bar() { char bar_buffer[64]; write(1, bar_buffer, 64); }
```

- Compiler optimizations can ruin your day:

```
int main() { foo(); bar(); }  
void foo() {  
    char foo_buffer[64];  
    read(open("/flag", 0), foo_buffer, 64);  
    memset(foo_buffer, 0, 64);  
}  
void bar() { char bar_buffer[64]; write(1, bar_buffer, 64); }
```